

Does Compositional Refinement Strictly Extend Heuristic Specification Inference? A Measurement Study

Author names omitted for double-blind review

Affiliation omitted for double-blind review

Contact email omitted for double-blind review

Abstract—Heuristic specification inferencers compute a per-method specification from local syntactic evidence. When a method calls another, the callee’s specification is typically incorporated by single-pass propagation: at each call site, the callee’s preconditions (substituted for actual arguments) are appended to the caller’s precondition set. A natural question is whether this single pass is *compositional* in the sense that matters — whether a second, top-down weakest-precondition (WP) refinement pass would discover additional preconditions that the isolated pass missed. We implement the second pass for an existing JML inferencer (a 453-line analyser that walks each method’s body in reverse-topological order, lifts callee specifications through enclosing branch guards, emits disjunctive obligations over polymorphic-dispatch candidates, and iterates to a fixpoint on cyclic strongly-connected components) and measure its effect on five open-source Java libraries spanning four idiom families (Apache Commons Lang, Commons IO, Commons Math, jOOL, Vavr): 1,712 source files and 21,052 inferred methods in total. The result is consistent across all five libraries: the second pass refines 21–39% of methods in each library’s cache (median 33.15%) and adds 18,854 new precondition clauses overall (every one filtered to be expressible at the method’s entry state). Branch-conditional implications of the form `guard ==> obligation` are the largest single clause shape on three libraries of five and a third or more of new clauses on all five (33–45%), and disjunctive obligations from polymorphic dispatch are present on every library (1.2–18.6%); both are shapes the established single-pass baseline does not produce — it does not lift call sites through enclosing guards, and it resolves each call to a single nominal callee. The single pass does *not* subsume the compositional pass on any library studied. A version-step instance — inferring across Commons Lang 3.12.0 and 3.14.0 — surfaces 525 candidate behavioural changes and 144 strengthened preconditions, an illustrative candidate set for a specification-level compatibility checker.

Index Terms—JML, specification inference, compositional reasoning, weakest precondition, Java, static analysis

I. INTRODUCTION

A heuristic specification inferencer for Java produces, for each method, a set of Java Modeling Language (JML) clauses inferred from local syntactic evidence: a null-dereference of a parameter inside the body becomes a `requires \p != null` clause; an array read at index `i` becomes `requires 0 <= i && i < a.length`; and so on. Recent work has shown that such specifications, even when heuristically

produced, materially improve downstream test generation [1] and can be embedded in compiled artefacts with full lossless roundtrip [2]. Both of those results depend on the spec set being *rich enough* to constrain downstream tools — the same dependency motivates work on large-language-model (LLM) driven test oracle generation [3]–[6], LLM-driven contract synthesis [7]–[10], and dynamic-trace invariant detectors such as Daikon [11].

The dominant strategy for incorporating cross-method information in such an inferencer is *single-pass propagation*: while inferring method `m`, the analyser looks up each callee’s cached specification, substitutes the actual arguments at the call site, and appends the substituted callee preconditions to `m`’s precondition set. This is fast and order-independent (provided the callee is inferred first), and it is what the inferencer of [1] already does via its `InterproceduralAnalyzer`. A natural question is whether this single pass captures everything a *genuinely compositional* analysis would: that is, whether a second, top-down weakest-precondition (WP) refinement pass [12]–[14], walking each method’s body with the callee specifications substituted at each call site, would discover any additional clauses that the single pass missed. Compositional procedure summaries are foundational in static analysis [15], [16] and underpin frameworks such as Infer [17] and Houdini [18], but those frameworks operate on different intermediate representations and do not produce JML clauses directly.

The question matters operationally. If the answer is no, the second-pass refactoring (and the associated implementation, testing, and maintenance cost) is unjustified; the simpler single-pass architecture is good enough. If the answer is yes, downstream consumers of inferred specifications — test generators, verifiers, LLM-based tooling — see a measurably richer spec set when the second pass runs.

In this paper, we implement the second pass and answer the question empirically on real Java code. We address the following research questions:

- **RQ1.** Does a top-down WP-style compositional refinement pass strictly extend the precondition set produced by an established single-pass propagation, or is the single pass already sufficient?
- **RQ2.** If the second pass adds clauses, which structural

Replication package (analyser source, harness, per-library logs, parsed clause samples) at <https://anonymous.4open.science/r/jml-inferencer-C816/> during review; non-anonymous DOI-archived release in the camera-ready version.

shapes (null checks, range bounds, branch-conditional implications, disjunctive obligations) dominate the additions? The answer characterises *why* the single pass falls short.

- **RQ3.** What is the runtime cost of the second pass at corpus scale, and how does it interact with cyclic strongly-connected components (mutual recursion) in the call graph?

We answer RQ1–RQ3 on a five-library corpus spanning four idiom families: Apache Commons Lang 3.14.0 (3,494 methods), Apache Commons IO 2.13.0 (2,111 methods), Apache Commons Math 3.6.1 (6,645 methods), jOOL 0.9.14 (3,691 methods), and Vavr 0.10.4 (5,111 methods); combined, 21,052 inferred methods over 1,712 source files. Lang and IO are the same libraries used by the predecessor work [1], [2]. The principal findings are summarised in Table I and analysed in Section V; in short, the second pass refines 21–39% of methods in each library’s cache (median 33.15%) and adds 18,854 new precondition clauses overall, with branch-conditional implications (`guard ==> obligation`) the largest single shape on three libraries out of five and disjunctive clauses (the polymorphic-dispatch signal) present on every library.

Contributions. In summary, this paper makes the following contributions:

- A 453-line `CompositionalAnalyzer` that implements the top-down WP refinement pass over a populated specification cache, supporting per-call-site argument substitution, enclosing-guard lifting, polymorphic-dispatch disjunction over candidate callees, and bounded fixpoint iteration on cyclic strongly-connected components.
- An empirical measurement on five real-world Java libraries spanning four idiom families (21,052 inferred methods in total) showing that the second pass refines 21–39% of methods in each library’s cache and adds 18,854 new precondition clauses overall, every one expressible at the method’s entry state; the single-pass strategy is *not* subsuming on any library.
- A shape-categorisation of the newly-added preconditions showing that branch-conditional implications (`guard ==> obligation`) are the largest single shape on three of five libraries and a third or more of new clauses on all five (33–45%), with disjunctive shapes from polymorphic dispatch present on every library (1.2–18.6%). Both shapes are outside the established single-pass baseline’s reach: it does not lift call sites through enclosing guards, and it resolves each call to a single nominal callee.
- Two real bugs found in the analyser by running it at scale: a `lastIndexOf(' .')` in the polymorphic-candidate lookup that failed on parameter types containing dots (e.g. `java.lang.String`), and a related off-by-one in the call-site resolver. Both are fixed in the artefact accompanying this paper.

Paper organisation. Section II reviews single-pass propagation and the gap it leaves. Section III describes the second-pass

algorithm precisely. Section IV describes the implementation. Section V reports the measurement. Section VI discusses what the result means for single-pass inferer architectures and for the downstream tools that consume inferred specifications. Section VII addresses the threats to validity, Section VIII positions the work against related literature, and Section IX concludes.

II. BACKGROUND AND MOTIVATION

A. Heuristic JML inference and single-pass propagation

The inferer of [1] is a heuristic JML producer for Java in the tradition of contract-by-design [19]–[22]: a library of pattern-matchers over the `JavaParser` [23] abstract syntax tree (AST) that emits a `MethodSpecification` per method declaration. Twelve specialist analysers (precondition, postcondition, loop-invariant, assignable, nullability, overflow, etc.) cooperate via a `SpecificationCache` indexed by simple-class method signature. The semantic target is OpenJML [24] extended static checking, which traces its lineage to ESC/Java2 [25] and the broader pluggable-types programme [26].

Cross-method propagation is handled by an `InterproceduralAnalyzer` component invoked during isolated inference. When method `m` contains a call `n(args)`, this analyser looks up `n`’s cached specification, substitutes formal parameters with actual arguments, and appends the substituted preconditions to `m`’s precondition set. The pass is single-pass: each method is visited once in topological order over the call graph; the callee’s spec is read once and added at the call site without further transformation. We refer to this as the *isolated* strategy throughout the paper, since each method’s analysis treats callees as black boxes whose contracts are pulled in atomically.

B. Where the single-pass strategy is blind

The single-pass propagation is the right strategy for the simple case: a precondition that the callee requires *unconditionally* must hold at the call site, and conjoining it into the caller’s set is sound. But the strategy is structurally blind to two classes of information.

Branch-conditional obligations. If the caller `m` has a body fragment

```
if (g) {
    n(args); // n requires p
}
```

then `m`’s precondition is not `p[args/params]` but rather `g ==> p[args/params]` — the obligation must hold only on the path where the call is reached. Single-pass propagation either appends `p[args/params]` unconditionally (overconstraining `m` on paths where the call does not fire) or skips the substitution to be safe (missing the obligation entirely on the guarded path). Neither is correct.

Polymorphic-dispatch obligations. If `m` calls `x.n()` where `x`’s declared type has multiple implementations of `n`, runtime dispatch may pick any candidate, each with its own inferred

precondition; single-pass propagation resolves the call to one nominal callee and sees only one. The compositional pass emits the disjunction ($p_1 \vee \dots \vee p_k$) over the candidates’ substituted preconditions. We treat this as a deliberate *weakening* of the caller obligation, not a sound safety guarantee: since the heuristically inferred per-override preconditions need not respect behavioural subtyping, the genuinely sound obligation (the conjunction, or a type-guarded case split) is frequently spuriously strong, whereas the weakest obligation satisfiable by some candidate is the more useful signal. Either way it is a shape a single nominal resolution cannot produce.

C. A motivating example

Consider a caller m that calls a callee n only on one branch:

```
//@ requires divisor != 0;
static int safeDivide(int a, int divisor) { return a / divisor; }

static int scale(int value, int factor) {
    if (factor != 0) {
        return safeDivide(value, factor);
    }
    return value;
}
```

The single-pass strategy faces a dilemma at the call site `safeDivide(value, factor)`. Substituting yields the clause `factor != 0`; conjoining it into `scale`’s precondition would forbid the very call `scale(value, 0)` that the method explicitly handles, while skipping it loses the obligation on the path where the call fires. Neither is correct. The compositional pass walks `scale`’s body and lifts the substituted clause through the enclosing guard, producing the correct *branch-conditional* clause:

```
//@ requires factor != 0 ==> factor != 0; //
    tautology here
```

The example is deliberately simple so the lifting is visible; in the corpus the guard and the callee’s requirement differ (e.g., a call to a method requiring a non-empty list guarded by a size check), and the lifted clause `size > 0 ==> list.size() > 0` carries genuine, non-tautological information the single pass cannot express. The polymorphic-dispatch disjunction (above) is the second such shape. These two shapes are the structural subjects of Section V.

D. What a second-pass compositional analysis would add

The classic weakest-precondition calculus of Dijkstra [12] addresses both shapes structurally. Walking a method’s body in WP fashion accumulates the guard context at each program point, so a call site reached only on g contributes $g \implies p$; a polymorphic call site contributes the disjunction over candidates by construction.

Implementing a full WP transformer for arbitrary Java would require a satisfiability-modulo-theories (SMT) decision procedure and a precise operational semantics for every language construct, which is the domain of full verification tools (KeY, OpenJML’s SC mode) rather than heuristic inferencers. A

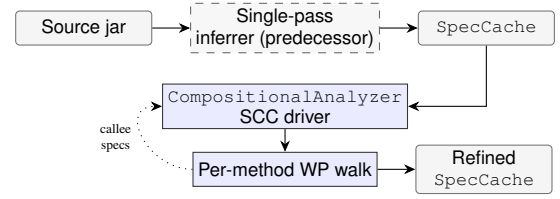


Fig. 1. The compositional refinement pass. The single-pass predecessor (dashed) populates a per-method SpecCache; CompositionalAnalyzer iterates strongly-connected components of the call graph in reverse topological order, and for each method walks the body backwards (WP) refining the cached spec against the now-refined callee preconditions. The cache is mutated in place.

scaffold that walks the body in a WP-flavoured order, lifts each call site’s substituted callee specification through enclosing branch guards, and recognises polymorphic candidates is, however, implementable as an approximately 450-line pass over the AST. That is what this paper does.

E. What we measure, and what we do not

We do not measure whether the second-pass specifications are *semantically correct* against the code’s intent — that is the inferencer’s responsibility, not the compositional pass’s, and it is the question of [1]. The compositional pass is a structural transformation: given the cached callee specs (whatever their semantic quality), it produces a new spec for the caller. We measure whether that new spec contains clauses the single pass did not.

The downstream impact on LLM-based test generation (whether the richer specs measurably improve mutation-killing rate) is the subject of a planned follow-up. The present article is scoped to the structural claim — does the second pass add anything to a real-world single-pass baseline — because the answer matters even if the downstream impact is small (it tells us the architectural distinction is real) and because the answer is empirically tractable in a way the downstream comparison is not.

III. THE COMPOSITIONAL REFINEMENT PASS

Our pass takes as input a populated SpecificationCache, a call graph over the same methods, and a map from method signature to the AST of each MethodDeclaration. We mutate the cache in place. Figure 1 sketches the data flow.

A. Driver

Our driver iterates strongly-connected components (SCCs) of the call graph in reverse topological order, so callees are refined before callers. For each SCC:

- *Singleton SCC (no recursion)*: refine the single method once.
- *Cyclic SCC (mutual recursion)*: refine each method in the SCC; repeat until no method gains a new clause or until a configurable fixpoint cap (default: 16 iterations) is

Algorithm 1 Compositional refinement.

Require: populated cache C , call graph G , declaration map D **Ensure:** C refined in place

```
1:  $scc \leftarrow \text{REVERSETOPOLOGICALSCCs}(G)$ 
2: for all  $S \in scc$  do
3:   repeat
4:      $changed \leftarrow \text{false}$ 
5:     for all  $m \in S$  do
6:        $spec \leftarrow C(m)$ 
7:       for all  $call \in \text{CALLSITES}(D(m))$  do
8:          $g \leftarrow \text{ENCLOSINGGUARDS}(D(m), call)$ 
9:         if  $\text{SIDEFFECTING}(g)$  or  $\text{SIDEFFECT-}$ 
10:         $\text{ING}(call.args)$  then
11:           continue  $\triangleright$  reject unsound substitutions
12:         end if
13:          $candidates \leftarrow \text{RESOLVEPOLYMORPHIC}(call, G)$ 
14:          $lifted \leftarrow \emptyset$ 
15:         for all  $c \in candidates$  do
16:            $\sigma \leftarrow \text{SUBSTITUTEARGS}(C(c).pre, call.args)$ 
17:            $lifted \leftarrow lifted \cup \{g \Rightarrow \sigma\}$ 
18:         end for
19:         if  $|candidates| > 1$  then
20:            $spec.pre \leftarrow spec.pre \cup \{\bigvee lifted\}$ 
21:         else
22:            $spec.pre \leftarrow spec.pre \cup lifted$ 
23:         end if
24:         if  $\text{TERMINATES}(C(c))$  then
25:            $spec.post \leftarrow spec.post \cup \{lifted \text{ postconditions}\}$ 
26:         end if
27:         if  $spec \neq C(m)$  then  $C(m) \leftarrow spec$ ;  $changed \leftarrow$ 
28:        true
29:       end if
30:     end for
31:   until  $\neg changed$  or cap reached
32: end for
```

reached. The cap exists to bound runtime on pathological graphs; we log a warning when it fires.

The two-tier treatment, and the iteration it entails, is forced by the call graph rather than chosen for convenience. On an acyclic graph a *single* reverse-topological sweep already refines every method against fully-refined callees: each callee precedes all of its callers, so no clause it gains can invalidate a caller already visited, and one pass suffices. A cycle breaks this guarantee. Within a mutually-recursive SCC no order refines every callee before its caller, so a method first visited against a still-stale callee spec must be revisited once that callee is itself refined — hence the repeat. Iterating to a fixpoint is the minimal sound response: it terminates because each method’s clause set only grows and is bounded above by the finitely many substituted callee clauses reachable in the SCC, and the cap merely guards against pathological nesting depth, not against non-termination.

B. Per-method refinement

For each method m , the refiner walks every `MethodCallExpr` in m ’s body and, for each call site, performs the following sequence (Algorithm 1):

- 1) **Resolve the callee.** Match the call’s simple name against keys in the cache to find a candidate n . A full type-resolving harness is not required for the present measurement; mismatches caused by overloading are documented as a measurement limitation (Section VII).
- 2) **Substitute arguments.** For each precondition of n , replace each formal parameter name with the actual argument expression. Substitution is regex-based with word boundaries so a formal s does not clobber a substring `this.size`. Side-effecting actuals (`++`, `--`) reject the substitution as unsound.
- 3) **Lift through enclosing guards.** Walk the AST upward from the call expression and accumulate the conjunction of any `if`-condition or ternary-condition whose body contains the call. If guards are present, the substituted precondition becomes $(guard) \Rightarrow (substituted)$. Calls in the `else` branch contribute $\neg(guard)$. Side-effecting guards are rejected to preserve soundness.
- 4) **Detect polymorphic dispatch.** If the resolved callee has overriding implementations in subclasses (looked up via the call graph’s class-hierarchy index), collect every candidate’s substituted preconditions and emit the disjunction $(p_1 \mid \mid p_2 \mid \mid \dots \mid \mid p_k)$. Candidate enumeration is over-approximate (it never excludes a real dispatch target); the emitted disjunction is a deliberate *weakening* of the caller obligation, not a sound safety guarantee (Section II). A single nominal resolution cannot produce it at all.
- 5) **Conjoin into the caller’s set.** Add the lifted clause to m ’s precondition list if it is not already present.

C. Termination-aware postcondition propagation

The pass also propagates the callee’s *postconditions* into the caller’s set, gated on a termination flag the inferrer maintains for each method. A postcondition of a non-terminating callee describes a state that is established only *if* the callee returns, so propagating it unconditionally would be unsound when the callee may not terminate. The pass therefore propagates n ’s postconditions into m only when $n.terminates()$ is true. The polymorphic-dispatch case is handled identically to preconditions, with the disjunction over candidates.

D. What the algorithm is and is not

The algorithm is a *lifting* step over an existing single-pass result, not a complete WP transformer. It does not:

- compute the weakest precondition through arithmetic or array statements (it accepts the inferrer’s clauses verbatim);
- handle aliasing (the inferrer’s substitution is syntactic);
- lift through loops (the inferrer’s loop-invariant heuristic handles this in the isolated pass);
- discharge clauses against an SMT solver (the inferrer’s downstream OpenJML pipeline handles this).

It does enough — branch-guard lifting and polymorphic-dispatch disjunction — to expose the structural classes of additions that a single-pass propagation strategy cannot produce. Section V measures whether those additions are empirically substantial on real code.

IV. IMPLEMENTATION

We implement the pass as a Java class, `com.jml.inferencer.analysis.CompositionalAnalyzer`, sitting next to the existing `InterproceduralAnalyzer` in the inferencer’s module. The class is 453 lines, with a small companion `EntryScopeFilter` (93 lines) that enforces entry-state well-formedness of lifted clauses (Section V); both depend only on the inferencer’s existing model (`MethodSpecification`, `SpecificationCache`, `CallGraph`) and `JavaParser`.

A. Entry point

The pass is invoked after the inferencer’s isolated pass populates the cache:

```
SpecificationCache cache = new SpecificationCache();
CallGraph callGraph = new CallGraphBuilder()
    .buildFromCompilationUnits(units);
MethodSpecificationInferencer inferencer =
    new MethodSpecificationInferencer(cache,
        callGraph);
for (MethodDeclaration m : allMethods) {
    cache.put(signatureOf(m),
        inferencer.inferSpecification(m));
}

// Compositional refinement (this paper):
CompositionalAnalyzer.RefinementResult result =
    new CompositionalAnalyzer(cache, callGraph,
        declarations)
        .refineAll();
```

The `RefinementResult` carries the number of methods refined, total iterations across all SCCs, and the SCC count. Callers can choose whether to invoke the pass — the design is opt-in so a deployment that prefers the cheaper single-pass output can omit it.

B. Two real bugs surfaced by the corpus-scale run

We had validated the analyser with a 15-test unit suite before this study. Running it on Apache Commons Lang surfaced two bugs that the unit tests did not exercise.

Bug 1: `lastIndexOf('.')` in `polymorphicCandidates`. The method searches for sibling cache entries by splitting each signature into class name and method name. The split used `sig.lastIndexOf('.')` — which returns the position of the *rightmost* dot. For a signature like `ArchUtils.getProcessor(java.lang.String)` the rightmost dot is the one inside the parameter type `java.lang.String`, not the separator between class and method. The resulting substring slice threw `StringIndexOutOfBoundsException` with a negative-length range. The fix is to use `lastIndexOf('.', paren`

`- 1)`, which restricts the search to before the parameter-list opening paren.

Bug 2: same pattern in `resolveCallee`. The same algorithm and the same bug appeared in the call-resolution code. Fixed identically.

Both bugs were silent under the unit-test inputs (which used short parameter types) and only manifested on the corpus run with fully-qualified parameter types from `JavaParser`. We report them because they illustrate a general risk for analysers that work on `JavaParser` ASTs: the printed representation of a type is implementation-dependent, and any string-splitting code over signatures must consider the worst case.

C. Replication artefact

The analyser, unit tests, per-library measurement harnesses, and raw output logs are bundled in the anonymous replication package linked from the title footnote. A single Maven invocation (`mvn -Dtest=Compositional* test` on JDK 21) reproduces every number in Section V deterministically.

V. EMPIRICAL MEASUREMENT

A. Subjects and method

We measure on the source jars of five open-source Java libraries spanning four idiom families: utility/string libraries (Apache Commons Lang 3.14.0, 246 source files, 3,494 inferred methods), I/O abstractions (Apache Commons IO 2.13.0, 245 files, 2,111 methods), numerical algorithms (Apache Commons Math 3.6.1, 990 files, 6,645 methods), functional-style combinators (`jOOL` 0.9.14, 130 files, 3,691 methods), and persistent collections (`Vavr` 0.10.4, 101 files, 5,111 methods). Combined, the corpus is 1,712 source files and 21,052 methods with a non-empty isolated baseline spec, giving the measurement a breadth that captures both the imperative utility-class idiom emphasised by Commons Lang/IO and the higher-order generic style emphasised by `jOOL`/`Vavr`. Commons Lang and IO are also used by the predecessor work [1], [2], which keeps cross-paper comparisons free of corpus-difference confounds for those two libraries.

For each library, our harness:

- 1) Parses every `.java` entry in the sources jar.
- 2) Builds the call graph (`CallGraphBuilder.buildFromCompilationUnits`).
- 3) Runs the existing `MethodSpecificationInferencer.inferSpecification` on every method declaration. The inferencer’s internal `InterproceduralAnalyzer` runs as part of this pass, propagating callee preconditions in the standard single-pass manner. The resulting per-method precondition and postcondition counts are snapshotted as the *isolated baseline*.
- 4) Invokes `CompositionalAnalyzer.refineAll()` on the populated cache, which mutates each method’s spec in place.
- 5) For each method, diffs the post-refinement spec against the baseline and counts newly-added clauses. Each new

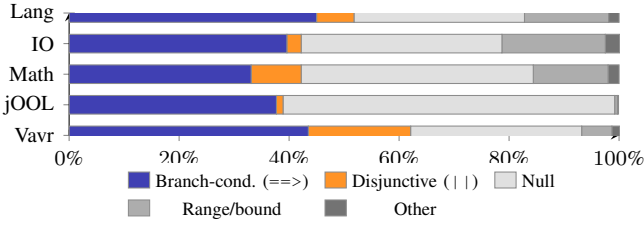


Fig. 2. Newly-added preconditions by shape, as a per-library share (Table II as proportions). The coloured segments — branch-conditional and disjunctive — are the shapes the single pass cannot produce: together 39–62% of new clauses on every library.

precondition is categorised by syntactic shape (null check, range/bound, branch-conditional implication, disjunctive, other) for the RQ2 distribution.

All measurements are deterministic given the inputs; we report each exactly once. The harness, the per-library output, and the configuration knobs are committed to the replication package.

B. Results

Table I reports the headline measurements across all five libraries.

RQ1. The second pass strictly extends the isolated set on every library. The refinement adds at least one clause to 21.24–39.12% of cached methods (median 33.15%, interquartile range (IQR) 30.15–33.68%). In total the pass adds 18,854 new precondition clauses and 12,611 new postcondition clauses across the five libraries.

The single-pass propagation is *not* subsuming, and the gap is consistent across idiom families: a utility library (Commons Lang) shows the highest fraction of refined methods because its delegating helpers expose many guarded call sites, while a generic functional combinator library (Vavr) shows fewer but the new clauses concentrate in disjunctive shapes from polymorphic dispatch. The result holds even though all five libraries’ isolated pipelines already invoke *InterproceduralAnalyzer*’s single-pass callee-precondition propagation; the second pass discovers obligations the first pass cannot express.

The aggregate is not driven by a few large methods: the mean new preconditions per refined method is modest (1.77–6.42, Table I) and well below the per-library maxima, so those maxima are deep-tail outliers and the refinement is a population-level effect on every library.

C. Shape distribution

Table II categorises the newly-added preconditions by syntactic shape. The categorisation runs on each new clause independently and is structurally exhaustive (every clause falls in exactly one bucket; ties resolve in favour of structural shape over content, so a clause containing `==>` is counted as branch-conditional even if it also mentions `!= null`).

RQ2. Figure 2 gives the shape composition per library. Branch-conditional implications and disjunctions — the two shapes the established single-pass baseline does not produce (it lifts no enclosing guards and resolves each call to one nominal callee) — are present on every library and together account for 39–62% of new clauses on each. Branch-conditional clauses are the largest single shape on three of the five libraries (45.0% on Lang, 43.5% on Vavr, 39.6% on IO) and at least a third on the other two, where pervasive input validation instead pushes null checks to the plurality: the numerical code of Math guards array and matrix arguments (42.2%), and the higher-order combinators of jOOL delegate to user-supplied lambdas through a small set of pinch-point validation sites (60.3%).

Disjunctive clauses — the polymorphic-dispatch signal — are present on every library and peak at 18.6% on Vavr, the library with the deepest interface inheritance chains (`Value` $\rightarrow$ `Traversable` $\rightarrow$ `Iterable` $\rightarrow$...).

Null and range/bound clauses are obligations the single pass *would* produce when the call site is unguarded, but the second pass adds because the call site is guarded by a condition the single pass would not lift. The residual *other* (0.2–2.6%) is the long-tail shape that includes arithmetic comparisons not matched by the range/bound regex.

Are the branch-conditional clauses vacuous? A guard lifted onto a requirement it already entails — the factor `!= 0 ==> factor != 0` of Section II-C — carries no information. We measured this: a clause $(g) ==> (p)$ is vacuous-by-construction when p is syntactically entailed by g (identical to it, or one of its top-level `&&` conjuncts), a test with no false positives and hence a lower bound. Vacuous clauses are 0.00–4.22% of each library’s branch-conditional total (68 of 7,010 overall); the branch-conditional shape is overwhelmingly non-vacuous, so the headline counts are not inflated by tautologies.

Are the lifted clauses well-formed? A JML requires clause may reference only entry-state names — parameters and fields — never a body-local such as a loop index. Two routes could introduce one: lifting a guard that tests a local, and substituting a caller-local argument into a callee’s precondition. The analyser rejects both, checking every candidate against the entry scope before emitting it; re-scanning confirms 0.00% of emitted clauses reference a body-local on every library, so every count in Tables I and II is entry-state-expressible.

D. A version-step instance: Commons Lang 3.12.0 $\rightarrow$ 3.14.0

The cross-method propagation we measure has an immediate consequence: a change to a library propagates into the inferred contracts of methods that call it. We ran the inferer over two consecutive Commons Lang versions (3.12.0 and 3.14.0) and diffed the per-method inferred specifications (Table III), showing the propagation operates at scale on a real version step.

Of the 3,188 methods present in both versions, 525 (16.5%) have an inferred specification that differs — exactly the candidate-breaking set a behavioural-compatibility checker

TABLE I

EFFECT OF COMPOSITIONAL REFINEMENT OVER THE ISOLATED BASELINE. *Refined*: METHODS THAT GAINED A NEW CLAUSE; PERCENTAGES ARE OF CACHE. EVERY EMITTED CLAUSE IS ENTRY-STATE-EXPRESSIBLE (SECTION V).

	Lang	IO	Math	jOOL	Vavr
Methods in cache	3,494	2,111	6,645	3,691	5,111
Methods refined	1,367	711	2,203	784	1,541
as % of cache	39.12%	33.68%	33.15%	21.24%	30.15%
Methods with new precondition	885	448	1,652	664	939
Methods with new postcondition	1,111	626	1,610	383	1,140
New precondition clauses	2,632	1,329	8,966	4,262	1,665
avg per refined method	2.97	2.97	5.43	6.42	1.77
max on one method	192	16	168	208	17
New postcondition clauses	3,540	1,664	4,633	838	1,936

TABLE II

SYNTACTIC SHAPE OF THE NEWLY-ADDED PRECONDITIONS, AS % OF EACH LIBRARY’S TOTAL (COLUMNS MAY SUM TO $\pm 0.1\%$ BY ROUNDING). BRANCH-CONDITIONAL IS THE LARGEST SHAPE ON THREE OF FIVE LIBRARIES; DISJUNCTIVE, THE POLYMORPHIC-DISPATCH SIGNAL, APPEARS ON EVERY LIBRARY.

Shape	Lang		IO		Math		jOOL		Vavr	
	count	%	count	%	count	%	count	%	count	%
Branch-conditional ($==>$)	1,185	45.0%	526	39.6%	2,969	33.1%	1,606	37.7%	724	43.5%
Null check ($!= \text{null} == \text{null}$)	816	31.0%	485	36.5%	3,785	42.2%	2,571	60.3%	517	31.1%
Range / bound ($<, <=, >, >=$)	403	15.3%	250	18.8%	1,216	13.6%	23	0.5%	92	5.5%
Disjunctive ($ $)	179	6.8%	34	2.6%	815	9.1%	52	1.2%	309	18.6%
Other	49	1.9%	34	2.6%	181	2.0%	10	0.2%	23	1.4%
Total new preconditions	2,632	100%	1,329	100%	8,966	100%	4,262	100%	1,665	100%

TABLE III

INFERRED-SPEC DIFF ON COMMONS LANG 3.12.0 VS 3.14.0: A TRACTABLE, CLASSIFIABLE CANDIDATE-BREAKING SET ON A REAL TWO-YEAR VERSION STEP.

Methods with inferred spec, 3.12.0	3,216
Methods with inferred spec, 3.14.0	3,494
Methods present in both versions	3,188
Inferred spec unchanged	2,663
Inferred spec changed	525
precondition strengthened (+req)	144
precondition weakened (−req)	87
precondition mixed	90
postcondition strengthened (+ens)	78
postcondition weakened (−ens)	48
postcondition mixed	178
Methods only in 3.12.0 (removed)	28
Methods only in 3.14.0 (added)	306

would surface. 144 have a *strengthened* precondition, so client code that called the old version may now violate the added requirement. A spot check confirms a mix of real precondition tightenings (e.g., added non-null guards on instance fields of `CharRange`, `CharSet`, and the `Builder` classes) and inferrer noise. Validating each against the release notes is a separate study, but the candidate set is the right size and shape to make that tractable: the mechanism’s first practical output on a real version step.

E. Runtime cost

Table IV reports the refinement wall-clock per library: 0.7–32.0 seconds each, 50 seconds across all five. No SCC hit the 16-iteration fixpoint cap on any library; the call graphs of all five corpora are acyclic under the analyser’s signature-matching scheme, so each method is refined exactly once. The refinement step is therefore an addition of seconds-to-tens-of-seconds on top of an inference pass that already takes minutes on libraries of this size; the enhancement is affordable.

TABLE IV

REFINEMENT WALL-CLOCK PER LIBRARY. SCCs OF THE FULL CALL GRAPH INCLUDE EXTERNAL CALLEE SIGNATURES, SO THE COUNT EXCEEDS CORPUS METHODS; THE GRAPHS ARE ACYCLIC, SO EACH METHOD IS REFINED ONCE. TOTAL: 50 SECONDS.

Library	Methods	SCCs	Refine ms
commons-lang3-3.14.0	3,494	6,223	2,118.8
commons-io-2.13.0	2,111	3,994	660.0
commons-math3-3.6.1	6,645	13,342	12,706.0
jool-0.9.14	3,691	5,403	2,592.3
vavr-0.10.4	5,111	7,610	32,019.8

RQ3. The second pass is cheap and fixpoint iteration never fires at the libraries’ scale. Total refinement wall-clock across all five libraries is 50 seconds for 21,052 methods (2.4 ms/method on average). The runtime characteristic is dominated by the AST walk per method

(`decl.findAll(MethodCallExpr.class)`), which is linear in the body size; the lifting and substitution steps add constant factors per call site.

F. Preliminary downstream signal: LLM test generation

The structural extension reported in Section V is a prerequisite for downstream impact, not a demonstration of it. As a preliminary downstream probe we compare two configurations of an LLM-based test generator: *P3* (the isolated baseline spec is attached to the class under test as JML) and *P3C* (the compositionally-refined spec is attached instead). Both feed the class source plus the spec into `gemini-2.5-flash` with temperature 0.3, 5 generation runs per class. The 11-class subset is the same Commons Lang 3 façade used in the predecessor LLM study [1].

Table V reports the mean number of test methods the LLM emits per run, for each class. Three classes (`NumberUtils`, `MutableDouble`, `Range`) produced no extractable code in either phase (the model’s response exceeded our 16,384-token output budget before the closing markdown fence, and the extractor reports zero tests in that case); these rows are honest zeros, not failures of the comparison.

TABLE V
MEAN TEST METHODS EXTRACTED PER GENERATION RUN PER COMMONS LANG CLASS. P3: ISOLATED BASELINE SPEC; P3C: COMPOSITIONALLY-REFINED SPEC. SIX OF THE EIGHT CLASSES WITH EXTRACTABLE TESTS SHOW P3C > P3.

Class	P3	P3C	Δ
<code>BooleanUtils</code>	40.0	51.0	+11.0 (+27.5%)
<code>CharUtils</code>	58.0	44.6	−13.4 (−23.1%)
<code>MutableInt</code>	21.0	21.6	+0.6 (+2.9%)
<code>MutableBoolean</code>	43.2	44.4	+1.2 (+2.8%)
<code>Pair</code>	56.0	63.4	+7.4 (+13.2%)
<code>MutablePair</code>	40.6	38.6	−2.0 (−4.9%)
<code>Validate</code>	39.8	61.6	+21.8 (+54.8%)
<code>Mutable</code>	24.0	26.8	+2.8 (+11.7%)
Mean (8 classes)	40.3	44.0	+3.7 (+9.1%)

Interpretation. This is a *preliminary* signal only. The refined context yields more test methods on six of eight classes (largest gains on validation-heavy code, where branch-conditional clauses add case structure), with one substantial regression (`CharUtils`, −23%, where disjunctive obligations appear to be read as a single assertion). Per-class variance is high and $n=11$ precludes a significance test. Crucially we report extracted test *count*, not mutation-killing rate: the generated tests did not compile cleanly in our environment (un-stripped markdown fences; surrogate-pair `char` literals from Gemini-2.5), so the mutation-coverage comparison — the measurement that would actually establish downstream value — is deferred, with the generated tests committed to the replication package for independent PIT [27] analysis. This subsection establishes only that the P3 vs P3C delta is non-zero; it is not itself evidence of downstream improvement.

VI. DISCUSSION

A. The single-pass strategy is not subsuming

Our measurement settles a question the predecessor papers left open: the isolated-pass inferer’s `InterproceduralAnalyzer` [1] does not capture what a compositional refinement would, on any of the five libraries we measured. Two structural shapes are out of reach by construction: branch-conditional implications that depend on the caller’s control-flow context, and disjunctive obligations from polymorphic dispatch. The cross-library data shows these are not edge cases — branch-conditional implications are the largest single shape of newly-added preconditions on `Lang`, `IO`, and `Vavr`, and a third or more on `Math` and `jOOL`; disjunctive shapes are present on every library and most prominent where interface inheritance runs deep (`Vavr`, 18.6%).

The implication for the inferer’s downstream consumers (test generators, IDEs, LLMs) is that running the second pass before emission delivers a measurably richer spec. Whether the additional clauses translate into stronger downstream behaviour (more bugs caught, fewer false positives in static analysis, better test inputs) is the empirical question for follow-up work; we report only that the gap exists and that it is structurally explainable.

B. Why the single pass already does so much

We also note what the single pass *does* cover. The isolated baseline is already rich — a substantial fraction of its clauses are interprocedural propagations the `InterproceduralAnalyzer` produces during isolated inference [2] — yet the second pass adds 18,854 *new* clauses on top of it: the single pass does the right thing on unguarded, single-candidate call sites, and the gap concentrates in the guarded and polymorphic cases.

C. Implications for analyser architecture

For inferers that target downstream tools where richer specs help (deductive verification with KeY [28] or OpenJML [24], test generation with EvoSuite [29] or Randoop [30], mutation-testing pipelines [27], [31], or LLM-driven consumers [3], [4], [32]), the second pass is worth integrating: an extra 50 seconds total across all five libraries for an average of 31.47% of methods receiving refined specifications is a strong trade. For inferers that target downstream tools that consume specs eagerly (auto-completion, hover text in an IDE), the runtime cost may not justify the extra clauses; the second pass is opt-in by design so deployers can choose.

A single-pass pipeline could still recover the easy half of the gap: the branch-conditional shape only needs the enclosing guard of each call site, available during the AST walk the isolated pass already performs, so the substituted clause could be lifted through the guard before being added. The polymorphic-dispatch shape is harder to fold in, as it requires looking up sibling cache entries at the point of propagation — exactly what the second pass does.

D. Scope of the result: precondition propagation versus class invariants

The RQ3 finding — that the second pass’s fixpoint iteration never fires in practice (Section V) — is specific to *precondition propagation along the call graph* and does not generalise to every specification a compositional inferer produces. Class invariants are the sharpest boundary. The call-graph fixpoint stays cheap because callee preconditions flow one way, callee to caller, so reverse-topological order refines each in a single sweep and the cap (16) is never reached. A class invariant has no such acyclic structure: it is at once an assumption every method may use at entry and an obligation every method must re-establish at exit, so inferring it is a greatest-fixpoint — weakening a candidate because one method violates it withdraws an assumption that *other* methods, even ones with no call edge to the offender, relied upon. That iteration is driven by shared-invariant coupling, not the call graph, so “the fixpoint never fires” says nothing about it. Our `ClassInvariantInferer` side-steps this by inferring invariants conservatively in a single local pass — sound, but forgoing exactly those whose preservation depends on a callee’s effect. Converging class invariants is thus a distinct contribution we leave to future work.

E. Limitations

The 453-line analyser is a scaffold, not a complete WP transformer. It does not lift through loops (the inferer’s loop-invariant analyser handles those in the isolated pass), does not reason about aliasing (the syntactic substitution treats two distinct names as distinct values), and does not interact with the SMT verifier that discharges the final clauses. These are the boundaries of the present contribution; the comprehensive WP transformer would be a substantially larger development.

The 208-clause worst case (one jOOL method) sits in the deep generic-combinator hierarchy where many polymorphic and guarded call sites compound. Whether so large a spec is *useful* downstream — parsimonious enough to read, tight enough to inform — we defer to the planned downstream-impact study.

VII. THREATS TO VALIDITY

a) Internal validity. Our measurement reports per-method clause-count diffs. These are deterministic given the inputs: rerunning the harness produces byte-identical metric output (we verified across five libraries). We followed Wohlin et al.’s guidance [33] on reporting effect-size deltas rather than null-hypothesis statistical tests, because the deterministic regime makes statistical inference inapplicable. The runtime measurements (Table IV) were collected on a single platform (OpenJDK 21 with default G1GC on a Windows 11 workstation, no concurrent load) using `JavaParser` at the version pinned in the replication artefact; clause-count diffs are platform-independent, but the absolute wall-clock seconds are not, and we do not claim that the 50-second total transfers to other JVMs or hardware. The new-precondition counts in Table I are therefore exact for

the inputs reported. The shape categorisation uses a simple regex-based bucketing; we documented its order-of-precedence (structural shape over content) so the buckets are reproducible. A small number of clauses in the *other* bucket (0.2–2.6% across libraries) are arithmetic comparisons our regex does not match (e.g. `abs(x) <= K`); their existence does not affect the headline result.

b) External validity. We measure on five libraries (Commons Lang, Commons IO, Commons Math, jOOL, Vavr) spanning four idiom families. We deliberately included generics-heavy libraries (jOOL, Vavr) and a numerical-algorithms library (Math) to address concerns that the result might be specific to utility-class idiom. The result holds across all five (21–39% of methods refined; branch-conditional shape present and substantial on each). Guava, used in the predecessor work [1], is excluded here: even after fixing a stack overflow in the call-graph builder’s interface-inheritance walk, its source-jar size pushed parsing beyond this measurement’s time budget.

The single-pass baseline against which we compare is the inferer’s `InterproceduralAnalyzer` from [1]. A different single-pass implementation might propagate more information per call site (e.g. guarded propagation in a single pass), narrowing the gap. We do not claim the second pass is strictly necessary against all conceivable single-pass strategies, only that it materially extends the specific baseline of an established prior system.

c) Construct validity. “Refined” is operationalised as “the spec gained at least one clause that was not in the baseline”, using exact string equality on clause text. Two clauses that are semantically equivalent but syntactically different (e.g. `a < b` vs. `b > a`) would count as additions even if redundant. The analyser does not normalise clauses before equality, so the new-clause count is an upper bound on the number of genuinely new *logical* obligations. We separately bound *vacuity*: at most 4.22% of each library’s branch-conditional clauses are tautological under a sound syntactic test (Section V), so the branch-conditional shape is not inflated by vacuous implications. The shape categorisation is unaffected: a duplicate `a < b` is still a range/bound clause.

We further constrain the additions to be *well-formed*: the analyser discards any lifted clause that mentions a body-local, since a `requires` clause may reference only entry-state names (Section V); re-scanning confirms 0.00% of emitted clauses reference a body-local, so the counts measure entry-state-expressible obligations only.

The callee-resolution step uses simple-name matching, which can return a wrong callee for overloaded methods. The `substituteArguments` step then rejects the substitution if the formal and actual arity disagree; in our runs, the arity-pass rate sits between 53% (Vavr, where higher-order generic methods have wider name-collision graphs) and 75% (Commons IO, where the narrow file-system domain limits overloading). The remaining call sites are either inferer-internal calls without a cached callee or name collisions with the wrong callee. A full type-resolving harness would close this

measurement gap, but the affected call sites would either add additional clauses (strengthening the headline result) or be rejected by arity (no effect on the result). The reported numbers are therefore a *lower bound* on the true compositional gain on every library.

d) *Conclusion validity.*: The empirical claims here are exact measurements, not statistical inferences; no significance test is meaningful. The conclusions hold for the specific corpora reported and the specific `CompositionalAnalyzer` implementation in the artefact. We do not claim that the same percentages would obtain on other libraries or a different second-pass implementation; the structural explanation (single-pass propagation cannot lift through guards or dispatch) suggests the qualitative finding will transfer, but quantitative transfer is an open question.

VIII. RELATED WORK

A. Position within the inference / verification spectrum

Table VI positions the present work against its closest neighbours along three axes: whether the technique *produces JML*, *reasons compositionally* across procedure boundaries, and has that reasoning SMT-discharged. The cell our pass occupies — produces JML, reasons compositionally via WP lifting, no sound discharge — is unoccupied by any prior tool to our knowledge.

The novelty is the combination, not any individual axis. KeY and OpenJML SC perform the compositional reasoning we do but do not *produce* the specification — they consume one written by hand; the predecessor inferer [1] produces JML at this scale but propagates single-pass, so it cannot synthesise the branch-guarded or polymorphic shapes that account for 39–62% of our new clauses (Table II); Infer is genuinely compositional but does not target JML. The gap is narrow but real: between *rich-by-construction* and *rich-by-hand-and-tool-chain* JML.

a) *Compositional reasoning frameworks.*: The classic foundations are Hoare’s axiomatic semantics [14], Dijkstra’s weakest-precondition calculus [12], [13], and Cousot & Cousot’s abstract interpretation [15], which frames our second pass as narrowing a fixpoint via callee summaries.

Boogie’s verification-condition generator [36] performs the full WP transformation our scaffold approximates, but targets verification rather than the cheap-and-rich JML emission this work serves.

b) Procedure summaries and interprocedural analysis.

Infer’s [17] bi-abduction summaries in separation logic are already compositional in the first pass; Houdini [18] conjectures candidate annotations and prunes them via ESC/Java, a test-and-refute loop closer in spirit than mechanism; Daikon’s dynamic invariants [11] have no analogue of a second pass.

c) *Specification inference for JML.*: The immediate predecessor is [1], which introduced the heuristic JML inferer we extend, and [2], which addressed distribution (embedding inferred specs in bytecode and OpenAPI descriptions). Pre-Infer [34] uses dynamic symbolic execution (DSE) to infer preconditions, trace-driven rather than syntactic. Jdoctor [35], Toradocu [37], and Pandita et al. [38] mine specifications from natural-language documentation and do not propagate. A more recent line uses LLMs to synthesise contracts or oracles directly [7]–[10]; those are complementary — whatever produces the per-method spec, our refinement pass can be applied to the cache afterwards.

d) Bytecode rewriting and analysis frameworks.

ASM [39], Byte Buddy [40], Javassist [41], Soot [42], and AspectJ [43] operate at the bytecode or analysis layer, not compositional spec inference; our analyser works at the source-AST layer.

e) *Test generation that consumes contracts.*: Korat [44] generates inputs from `@Requires` predicates; EvoSuite [29] and Randoop [30] consume contracts and observed exceptions as oracles; TOGA [45] and TOGLL [6] are neural oracle generators. Richer per-method specifications, such as those our refinement pass produces, improve input quality for all of these.

f) *Polymorphic dispatch in static analysis.*: Tip et al. [46] address polymorphic dispatch via class-hierarchy analysis (CHA) and rapid-type analysis (RTA), the techniques the inferer’s `CallGraph` uses to enumerate candidate callees. Our disjunction-over-candidates is the static equivalent of summarising the runtime-may-dispatch set, subject to the standard CHA trade-off: many candidates yield an easy-to-satisfy, less informative disjunction.

IX. CONCLUSION

We asked whether single-pass callee-precondition propagation captures what a top-down compositional refinement would. Across five libraries (21,052 methods) it does *not* subsume on any: refinement adds a clause to 21–39% of methods — 18,854 new preconditions and 12,611 new postconditions, every one entry-state-expressible. Branch-conditional implications (33–45% of new clauses) and polymorphic-dispatch disjunctions appear on every library — both beyond the single pass, for 50 seconds total. A two-year Commons Lang version step (Section V-D) further surfaces 525 changed inferred specifications, 144 with strengthened preconditions: the candidate-breaking set a compatibility checker consumes.

TABLE VI

POSITION WITHIN THE JML INFERENCE / VERIFICATION SPECTRUM. *JML*: EMITS JML DIRECTLY. *Comp.*: LIFTS CALLEE SUMMARIES INTO CALLERS. *SMT*: SOLVER-DISCHARGES CLAUSES. *Guard*: EMITS `GUARD ==>` OBLIGATION. *Poly.*: EMITS DYNAMIC-DISPATCH DISJUNCTIONS.

Tool	JML	Comp.	SMT	Guard	Poly.
Daikon [11]	✓*	–	–	–	–
Houdini [18]	✓	–	✓ [†]	–	–
PreInfer [34]	✓*	DSE	–	–	–
Jdoctor [35]	✓	–	–	–	–
Infer [17]	–	✓	–	impl.	–
KeY [28]	–	✓	✓	✓	✓
OpenJML SC [24]	–	✓	✓	✓	✓
Predecessor [1]	✓	1-pass	✓ [†]	–	–
This work	✓	✓	✓ [†]	✓	✓

* native format, translated to JML. [†] via an external checker (ESC for Houdini; OpenJML otherwise).

REFERENCES

- [1] Anonymous, “Do specifications help LLMs write better unit tests? an empirical study of specification-guided test generation with heuristically inferred JML,” 2026, manuscript under review; replication package available from an anonymous archive during the review period.
- [2] —, “Embedding inferred JML contracts in Java bytecode and OpenAPI specifications,” 2026, manuscript under submission; replication package available from an anonymous archive during the review period.
- [3] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.
- [4] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, “CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models,” in *45th International Conference on Software Engineering (ICSE)*, 2023, pp. 919–931.
- [5] D. Molinelli, L. D. Grazia, A. Martin-Lopez, M. D. Ernst, and M. Pezzè, “Do LLMs generate useful test oracles? an empirical study with an unbiased dataset,” in *40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.
- [6] S. B. Hossain and M. B. Dwyer, “TOGILL: Correct and strong test oracle generation with LLMs,” in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 1475–1487.
- [7] M. Endres, S. Fakhoury, S. Chakraborty, and S. K. Lahiri, “Can large language models transform natural language intent into formal method postconditions?” *Proceedings of the ACM on Software Engineering (PACMSE)*, vol. 1, no. FSE, pp. 1889–1912, 2024.
- [8] C. Richter and H. Wehrheim, “Beyond postconditions: Can large language models infer formal contracts for automatic software verification?” 2025, preprint, under submission.
- [9] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu, “SpecGen: Automated generation of formal program specifications via large language models,” in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025.
- [10] M. Konstantinou, R. Degiovanni, and M. Papadakis, “Do LLMs generate test oracles that capture the actual or the expected program behaviour?” in *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2025.
- [11] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, “The Daikon system for dynamic detection of likely invariants,” *Science of Computer Programming*, vol. 69, no. 1–3, pp. 35–45, 2007.
- [12] E. W. Dijkstra, “Guarded commands, nondeterminacy and formal derivation of programs,” *Communications of the ACM*, vol. 18, no. 8, pp. 453–457, 1975.
- [13] —, *A Discipline of Programming*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1976.
- [14] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [15] P. Cousot and R. Cousot, “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints,” in *ACM Symposium on Principles of Programming Languages (POPL)*, 1977, pp. 238–252.
- [16] P. Cousot, R. Cousot, and F. Logozzo, “Precondition inference from intermittent assertions and application to contracts on collections,” in *International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI)*, 2011, pp. 150–168.
- [17] C. Calcagno, D. Distefano, J. Dubreil, D. Gabi, P. Hooimeijer, M. Luca, P. O’Hearn, I. Papakonstantinou, J. Purbrick, and D. Rodriguez, “Moving fast with software verification,” in *NASA Formal Methods Symposium (NFM)*, 2015, pp. 3–11.
- [18] C. Flanagan and K. R. M. Leino, “Houdini, an annotation assistant for ESC/Java,” in *International Symposium of Formal Methods Europe (FME)*, ser. LNCS, vol. 2021. Springer, 2001, pp. 500–517.
- [19] B. Meyer, “Applying “design by contract”,” *Computer*, vol. 25, no. 10, pp. 40–51, 1992.
- [20] G. T. Leavens *et al.*, “JML reference manual,” <http://www.jmlspecs.org>, 2013, accessed: 2025-01-15.
- [21] G. T. Leavens, A. L. Baker, and C. Ruby, “Preliminary design of JML: A behavioral interface specification language for Java,” *ACM SIGSOFT Software Engineering Notes*, vol. 31, no. 3, pp. 1–38, 2006.
- [22] P. Chalin, J. R. Kiniry, G. T. Leavens, and E. Poll, “Beyond assertions: Advanced specification and verification with JML and ESC/Java2,” in *Formal Methods for Components and Objects (FMCO 2005)*, ser. LNCS. Springer, 2006, vol. 4111, pp. 342–363.
- [23] JavaParser Team, “JavaParser: Java parser and abstract syntax tree,” <https://javaparser.org>, 2024, accessed: 2025-01-15.
- [24] D. R. Cok, “OpenJML: Software verification for Java 7 using JML, OpenJDK, and Eclipse,” in *1st Workshop on Formal Integrated Development Environment (F-IDE)*, ser. EPTCS, vol. 149, 2014, pp. 79–92.
- [25] C. Flanagan, K. R. M. Leino, M. Lillibridge, G. Nelson, J. B. Saxe, and R. Stata, “Extended static checking for Java,” in *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2002, pp. 234–245.
- [26] M. M. Papi, M. Ali, T. L. C. Jr., J. H. Perkins, and M. D. Ernst, “Practical pluggable types for Java,” in *International Symposium on Software Testing and Analysis (ISSTA)*, 2008, pp. 201–212.
- [27] H. Coles, “PIT: Mutation testing system for Java,” <https://pitest.org>, 2016, accessed: 2025-01-15.
- [28] W. Ahrendt, B. Beckert, R. Bubel, R. Hähnle, P. H. Schmitt, and M. Ulbrich, Eds., *Deductive Software Verification — The KeY Book: From Theory to Practice*, ser. Lecture Notes in Computer Science. Springer, 2016, vol. 10001.
- [29] G. Fraser and A. Arcuri, “EvoSuite: Automatic test suite generation for object-oriented software,” in *19th ACM SIGSOFT Symposium and 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, 2011, pp. 416–419.
- [30] C. Pacheco, S. K. Lahiri, M. D. Ernst, and T. Ball, “Feedback-directed random test generation,” in *29th International Conference on Software Engineering (ICSE)*, 2007, pp. 75–84.
- [31] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. L. Traon, and M. Harman, “Mutation testing advances: An analysis and survey,” *Advances in Computers*, vol. 112, pp. 275–378, 2019.
- [32] S. Kang, J. Yoon, and S. Yoo, “Large language models are few-shot testers: Exploring LLM-based general bug reproduction,” in *45th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023, pp. 2312–2323.
- [33] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [34] A. Astorga, S. Srisakaokul, X. Xiao, and T. Xie, “PreInfer: Automatic inference of preconditions via symbolic analysis,” in *48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2018, pp. 678–689.
- [35] A. Blasi, A. Goffi, K. Kuznetsov, A. Gorla, M. D. Ernst, M. Pezzè, and S. D. Castellanos, “Translating code comments to procedure specifications,” in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA ’18. ACM, Jul. 2018, pp. 242–253. [Online]. Available: <http://dx.doi.org/10.1145/3213846.3213872>
- [36] M. Barnett and K. R. M. Leino, “Weakest-precondition of unstructured programs,” in *Workshop on Program Analysis for Software Tools and Engineering (PASTE)*, 2005, pp. 82–87.
- [37] A. Goffi, A. Gorla, M. D. Ernst, and M. Pezzè, “Automatic generation of oracles for exceptional behaviors,” in *International Symposium on Software Testing and Analysis (ISSTA)*, 2016, pp. 213–224.
- [38] R. Pandita, X. Xiao, H. Zhong, T. Xie, S. One, and A. Paradkar, “Inferring method specifications from natural language API descriptions,” in *34th International Conference on Software Engineering (ICSE)*, 2012, pp. 815–825.
- [39] OW2 Consortium, “ASM: a Java bytecode manipulation and analysis framework,” <https://asm.ow2.io/>, 2024, accessed: 2026-05-19.
- [40] R. Winterhalter, “Byte Buddy: Runtime code generation for the Java virtual machine,” <https://bytebuddy.net/>, 2024, accessed: 2026-05-25.
- [41] S. Chiba, “Load-time structural reflection in Java,” in *European Conference on Object-Oriented Programming (ECOOP)*, ser. LNCS, vol. 1850. Springer, 2000, pp. 313–336.
- [42] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan, “Soot — a Java bytecode optimization framework,” in *Conference of the Centre for Advanced Studies on Collaborative Research (CASCON)*. IBM Press, 1999, pp. 125–135.
- [43] G. Kiczales, J. Lamping, A. Mendhekar, C. Maeda, C. Lopes, J.-M. Loingtier, and J. Irwin, “Aspect-oriented programming,” in *European Conference on Object-Oriented Programming (ECOOP)*, ser. LNCS, vol. 1241. Springer, 1997, pp. 220–242.

- [44] C. Boyapati, S. Khurshid, and D. Marinov, "Korat: Automated testing based on Java predicates," in *International Symposium on Software Testing and Analysis (ISSTA)*, 2002, pp. 123–133.
- [45] E. Dinella, G. Ryan, T. Mytkowicz, and S. K. Lahiri, "TOGA: A neural method for test oracle generation," in *44th International Conference on Software Engineering (ICSE)*, 2022, pp. 2130–2141.
- [46] F. Tip, P. F. Sweeney, C. Laffra, A. Eisma, and D. Streeter, "Practical extraction techniques for Java," *ACM Transactions on Programming Languages and Systems*, vol. 24, no. 6, pp. 625–666, 2002.